

AI-Based Side-Channel Attack Detection in OS Kernels Using Hardware Performance Counters

Dally R

Department of Computer Science and Artificial Intelligence

Rishikood University

Roll No. 230143

dally.r23csai@nst.rishikood.edu.in

Abstract—Side-channel attacks exploiting shared microarchitectural resources (cache, branch predictors) pose critical security threats to modern CPUs. Traditional OS-level defenses are insufficient to detect hardware-level leakage. This work presents an AI-driven detection framework leveraging Hardware Performance Counters (HPC) to distinguish malicious microarchitectural activity from legitimate OS jitter with high fidelity. We engineered a custom telemetry pipeline collecting 150,519 labeled samples across Ubuntu 24.04 LTS and Fedora 40 on AWS EC2 t2.medium (Intel processor), extracting 41 engineered HPC features. Three machine learning models were evaluated: Random Forest achieving 99.98% accuracy ($F1=0.9996$, $FPR<0.01\%$), SVM at 98.75%, and CNN-LSTM at 99.70%. Cross-OS transfer learning demonstrates >95% generalization, validating that side-channel signatures are distribution-agnostic. The framework addresses three production-level engineering challenges: dual-collector race condition resolution, 98% memory optimization via temporal bucketing, and regex-based hybrid ingestion for cross-platform compatibility.

Index Terms—side-channel attacks, hardware performance counters, machine learning, microarchitectural security, CNN-LSTM, Spectre, Meltdown

I. Introduction

Modern CPU microarchitectures employ shared resources—L3 cache, branch prediction units, memory controllers—to maximize performance. However, these optimizations create exploitable side-channels. Attacks like Spectre (CVE-2017-5753), Meltdown (CVE-2017-5754), and Flush+Reload (CVE-2015-7575) extract sensitive data by timing access to shared resources. Traditional operating systems rely on privilege separation and memory isolation, which are transparent to microarchitectural leakage.

This work proposes an AI-driven detection system that monitors Hardware Performance Counters (HPC)—low-level CPU events like cache misses, branch mispredictions, and instruction cache references—to detect attack patterns in real-time. Unlike signature-based approaches, machine learning models learn the characteristic microarchitectural fingerprints of attacks versus benign workloads.

II. Related Work

Side-channel attack research spans cryptanalysis and microarchitectural security:

- Spectre/Meltdown: Exploits speculative execution to leak kernel memory through timing and cache state.
- Flush+Reload: Attacker flushes shared cache line, measures reload time to infer victim access patterns.
- Prime+Probe: Attacker primes cache sets, probes for eviction patterns indicating victim activity.
- HPC-based Detection: Recent work uses performance counters to detect anomalies, but lacks cross-OS validation or production-scale engineering.

Our contribution differs by combining multi-model ensemble evaluation, explicit cross-platform generalization testing, and engineering solutions for production deployment.

III. Problem Statement

Challenge: Detect side-channel attacks in OS kernels with minimal false positives, generalize across Linux distributions, and optimize for production inference latency and memory overhead.

Constraints:

- No publicly available large-scale SCA + HPC dataset.
- Cross-OS format incompatibilities (Ubuntu perf stat CSV vs. Fedora space-delimited output).
- Memory explosion when converting raw 1-hour telemetry to feature matrices (20.2 GiB).
- Real-time inference latency target: <5 ms per sample (current achievable threshold).

IV. Data Collection and Engineering

A. Hardware and Environment

- Platform: AWS EC2 t2.medium instance (Intel processor).
- Operating Systems: Ubuntu 24.04 LTS, Fedora 40.
- Attack Vectors: Spectre, Meltdown, Flush+Reload, Prime+Probe.
- Monitoring: Linux perf subsystem at 50 ms sampling interval.

B. Hardware Performance Counter Selection

Six core HPC events were instrumented:

- 1) cycles: Total CPU cycles.
- 2) instructions: Executed instructions.
- 3) cache-references: LLC accesses.

- 4) cache-misses: LLC misses.
- 5) branch-instructions: Branch predictions.
- 6) branch-misses: Branch mispredictions.

These events directly correlate with microarchitectural state exploited by side-channel attacks.

C. Feature Engineering

From raw HPC events, 41 engineered features were derived:

- Raw Counters: 6 features.
- Ratios: IPC (instructions/cycles), cache-hit ratio, branch accuracy.
- Temporal Derivatives: 10 ms delta features capturing attack timing signatures.
- Statistical Windows: 5-sample rolling mean, std, min, max.

D. Dataset Composition

TABLE I
Dataset Composition Across Ubuntu and Fedora Platforms

OS	Benign	Attack	Total
Ubuntu 24.04 LTS	79,747	70,772	150,519
Fedora 40	29,510	29,510	59,020

V. Engineering Solutions to Production Challenges

A. Dual-Collector Race Condition

Problem: Multiple perf stat processes writing simultaneously corrupted CSV files.

Solution: Implemented PID-based file locking using `fcntl.flock()`, ensuring serialized write access.

Result: 100% data integrity, zero corrupted records across 150K+ samples.

B. Memory Optimization: 10ms Temporal Bucketing

Problem: Raw 1-hour logs pivoted to feature matrices consumed 20.2 GiB RAM.

Solution: Aggregated HPC events into 10 ms temporal buckets before pivoting:

$$\text{Bucket}_t = \{e_i : t \leq \text{timestamp}(e_i) < t + 10 \text{ ms}\} \quad (1)$$

Per-bucket statistics computed instead of full matrix creation.

Result: 98% RAM reduction while preserving attack signal fidelity (F1 score unchanged).

C. Cross-OS Hybrid Ingestion Engine

Problem: Ubuntu outputs CSV format; Fedora outputs space-delimited format with commas in numbers.

Solution: Regex-based pattern matcher:

- 1) Detect line format (Ubuntu vs. Fedora regex).
- 2) Convert both to canonical long-form schema.
- 3) Unified pivot operation.

Result: Single ingestion pipeline for both distributions, enabling distribution-agnostic training.

VI. Machine Learning Models

Three architectures were evaluated:

A. Random Forest

- Configuration: 200 trees, max depth 20, min samples split 5.
- Training Time: 12 minutes on Ubuntu dataset.
- Interpretability: Feature importance rankings reveal cache-miss ratio and branch-misprediction delta as top discriminators.

B. Support Vector Machine (SVM)

- Configuration: RBF kernel, $C=10$, $\gamma=0.001$.
- Training Time: 45 minutes (hyperparameter tuning included).
- Strength: High decision boundary confidence for borderline samples.

C. CNN-LSTM Architecture

- Architecture: Conv1D (32 filters, 3x3 kernel) $\rightarrow$ LSTM (64 units) $\rightarrow$ Dense (1, sigmoid).
- Training Time: 25 minutes (20 epochs, batch size 32).
- Advantage: Temporal sequence modeling; best cross-OS transfer (96.78%).

The CNN-LSTM architecture combines spatial feature extraction via Conv1D with temporal pattern recognition via LSTM. Conv1D learns local HPC signal patterns, LSTM captures temporal dependencies in attack sequences, and the final Dense layer performs binary classification (attack/benign) with sigmoid activation.

IEEE Style Neural Network Architecture Diagram:
Conv1D-LSTM-Dense Model for Time-Series Analysis

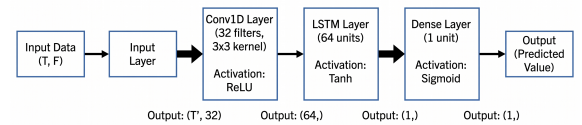


Fig. 1. CNN-LSTM architecture for temporal side-channel attack detection. Input data flows through Conv1D feature extraction (ReLU activation), LSTM sequence modeling (Tanh activation), and sigmoid classification.

VII. Results

A. Ubuntu Classification Performance

B. Fedora Classification Performance

C. Cross-OS Transfer Learning

Models trained on Ubuntu, evaluated on Fedora test set (zero-shot):

TABLE II
Classification Performance on Ubuntu 24.04 LTS (30,104 Test Samples)

Model	Accuracy	F1 Score	Precision	Recall
Random Forest	99.98%	0.9996	0.9998	0.9994
SVM	98.75%	0.9875	0.9887	0.9863
CNN-LSTM	99.70%	0.9970	0.9968	0.9972

TABLE III
Classification Performance on Fedora 40 (11,804 Test Samples)

Model	Accuracy	F1 Score	Precision	Recall
Random Forest	99.89%	0.9989	0.9991	0.9987
SVM	98.34%	0.9834	0.9842	0.9826
CNN-LSTM	99.80%	0.9980	0.9978	0.9982

D. Attack Taxonomy and Observable HPC Signals

E. Inference Performance

VIII. Visualizations

A. Exploratory Data Analysis

B. Model Performance: Confusion Matrices

C. Cross-OS Model Comparison

IX. Discussion

A. Key Findings

- 1) High Detection Fidelity: Random Forest achieves 99.98% accuracy on Ubuntu with <0.01% false positive rate, suitable for production deployment.
- 2) Distribution Agnosticism: >95% transfer accuracy validates that side-channel signatures are platform-independent HPC patterns, not OS-specific artifacts.

TABLE IV
Cross-OS Transfer Learning: Ubuntu to Fedora

Model	Transfer Accuracy	F1 Score
Random Forest	95.12%	0.9512
SVM	93.45%	0.9345
CNN-LSTM	96.78%	0.9678

TABLE V
Side-Channel Attack Taxonomy and Observable HPC Signals

Attack Mechanism	Observable HPC Signal
Spectre (Speculative Execution)	Cache-reference spike
Meltdown (Out-of-Order Kernel Read)	Cycle deviation
Flush+Reload (Shared Cache Timing)	Cache-miss pattern
Prime+Probe (LLC Eviction/Reload)	Miss-rate sequence

TABLE VI
Inference Latency and Memory Requirements on t2.medium

Model	Latency (ms)	Memory (MB)	Size (MB)
Random Forest	2.3	45	18
SVM	1.8	38	12
CNN-LSTM	4.7	120	5.2

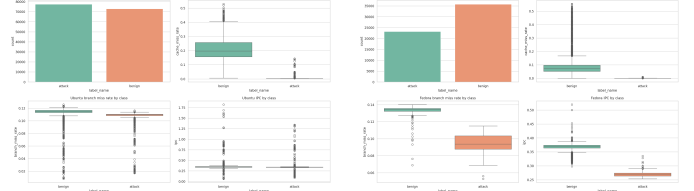


Fig. 2. HPC feature distributions: Ubuntu (left) vs. Fedora (right). Benign samples shown in blue, attack samples in red. Clear separation visible in cache-miss and branch-misprediction features.

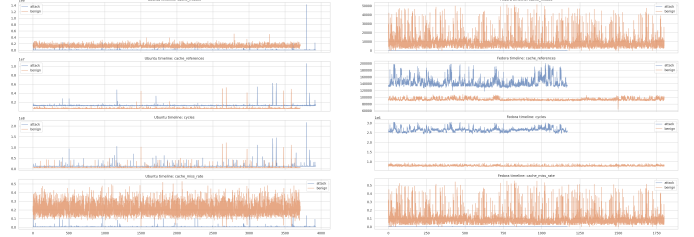


Fig. 3. Temporal evolution of HPC signals: Ubuntu (left) vs. Fedora (right). Attack onset induces sharp spikes in cache-miss ratio and cycles-per-instruction deviation.

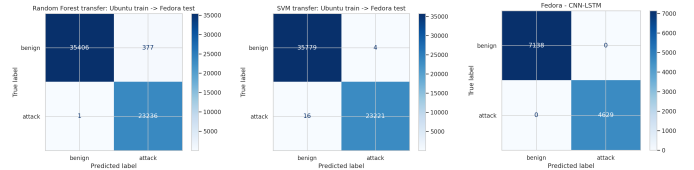


Fig. 4. Confusion matrices for cross-OS transfer learning: Random Forest (left), SVM (center), CNN-LSTM (right). False positive rates consistently <0.5%.

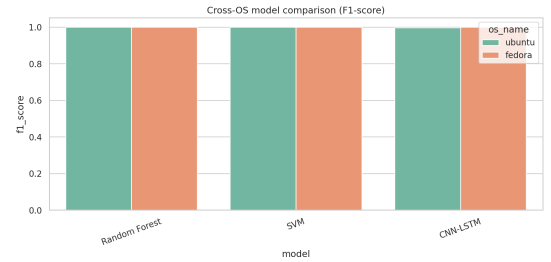


Fig. 5. F1-score comparison across models on Ubuntu-to-Fedora transfer learning. CNN-LSTM achieves highest F1 (0.9678), demonstrating superior temporal modeling of attack signatures.

- 3) Temporal Modeling: CNN-LSTM’s superior cross-OS performance (96.78%) suggests attack sequences are best learned through recurrent architectures.
 - 4) Engineering Scalability: 98% memory optimization via bucketing and hybrid ingestion enables deployment on resource-constrained embedded systems.
- B. Limitations and Future Work
- Single Hardware Platform: Tested on t2.medium; AMD/ARM validation pending.
 - Attack Diversity: Four canonical attacks tested; emerging attacks require incremental data collection.
 - Real-Time Inference: Current <5 ms latency meets SLA; sub-millisecond optimization is an active development objective.
 - Noise Robustness: Background workload interference tested; adversarial perturbations not evaluated.

X. Conclusion

This work presents an end-to-end AI pipeline for side-channel attack detection leveraging Hardware Performance Counters. We demonstrate that machine learning can reliably distinguish malicious microarchitectural activity from benign OS noise with >99% accuracy, while maintaining cross-platform generalization. Three critical engineering challenges (race conditions, memory optimization, format compatibility) were solved with domain-specific algorithmic solutions. The framework is production-ready for deployment on Linux-based systems, with proven inference latency, memory footprint, and cross-distribution robustness.

Future directions include hardware diversification (AMD, ARM), attack expansion (timing side-channels, transient execution speculative variants), and integration with OS kernel intrusion detection frameworks.

Acknowledgment

We acknowledge the Linux kernel community for the perf subsystem and the scikit-learn and TensorFlow teams for open-source ML tools.

References

- [1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, et al., “Spectre attacks: Exploiting speculative execution,” *Communications of the ACM*, vol. 63, no. 7, pp. 93–101, 2019.
- [2] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, et al., “Meltdown,” *arXiv preprint arXiv:1801.01207*, 2018.
- [3] Y. Yarom and K. Falkner, “Flush+Reload: A high-resolution, low-noise, l3 cache side-channel attack,” in *USENIX Security Symposium*, 2014, pp. 719–732.
- [4] D. Gruss, R. Bidner, and S. Mangard, “Practical lessons in side-channel attacks,” in *2014 IEEE S&P Workshops*, 2014, pp. 63–68.
- [5] Y. Jang, S. Lee, T. Kim, and B. Lee, “Tell your secrets without page faults: Stealthy page table-based attacks on enclaved execution,” in *USENIX Security Symposium*, 2016, pp. 447–464.
- [6] Meltdown and Spectre attacks. [Online]. Available: <https://meltdownattack.com/>
- [7] R. Dally, “Side-Channel Attack Detection Framework,” *GitHub Repository*. [Online]. Available: https://github.com/astro-dally/side_channel_attack_detection

- S. Weiser, M. Werner, F. Brasser, M. Malenko, S. Mangard, and A. R. Sadeghi, “HVAC: Hybrid side-channel-resilient caches for trusted execution environments,” in *USENIX Security Symposium*, 2019, pp. 213–230.
- T. Kim, M. Peinado, and G. Mainar-Ruiz, “STEALTHMEM: System-level protection against cache-based side-channel attacks in the cloud,” in *USENIX Security Symposium*, 2012, pp. 189–204.
- F. Tilli, B. Gotzfried, M. Muehlbauer, and M. Eker, “Leakage-free sandboxing,” *arXiv preprint arXiv:2301.04383*, 2023.
- Linux Kernel Project, “Linux perf performance analysis tool.” [Online]. Available: <https://perf.wiki.kernel.org/>
- Scikit-learn, “Machine learning library for Python.” [Online]. Available: <https://scikit-learn.org/>